Parte 1 — Complejidad algorítmica (Big O)

1.1 Clasifica la complejidad

#	Situación	Complejidad	Justificación
1	Buscar una palabra en un diccionario abriendo por la mitad.	O(log n)	Cada vez se descarta aproximadamente la mitad de las páginas que quedan, por lo que se necesitan pocas búsquedas incluso si el diccionario crece.
2	Revisar carnés uno por uno hasta encontrar el buscado.	O(n)	En el peor caso hay que revisar todos los carnés. Si hay el doble de carnés, potencialmente hay que revisar el doble.
3	Sacar la primera carta de un mazo ya barajado.	O(1)	Solo se realiza una acción: Tomar la carta de arriba, el número de cartas no cambia la cantidad de pasos.
4	Comparar cada estudiante con todos los demás para buscar cumpleaños iguales.	$O(n^2)$	Cada estudiante se compara con muchos otros. Al aumentar la cantidad de estudiantes, crece aproximadamente el número de pares que se revisan.
5	Consultar la hora en un reloj.	O(1)	Leer la hora requiere una cantidad constante de acciones, independientemente del tamaño de cualquier conjunto de datos.

1.2 De la vida real al análisis

Ejemplo: revisar una lista de tareas pendientes.

Leo cada tarea de una lista para comprobar si está completada y marcarla. Si la lista tiene más tareas, debo revisar cada una.

- **Complejidad:** O(n).
- Si la entrada se hace **10 veces más grande**, debo revisar aproximadamente 10 veces más tareas, por lo que el tiempo también **aumentaría unas 10 veces**.

1.3 ¿Cuál escalar mejor?

Recomendaría la Forma B: O(n log n) solo si no sé qué tan grande será **n** en el futuro.

Aunque para **n = 5** la Forma A puede ser más rápida por sus costos menores constantes, O(n log n) crece más lentamente que O(n²). Cuando la cantidad de datos aumenta mucho, la Forma B necesita considerablemente menos operaciones y será más conveniente.

1.4 Verdadero o falso

A. *Falso.* Un algoritmo O(1) no siempre tarda menos segundos que uno O(n). Big O describe el crecimiento del tiempo, pero no garantiza que las constantes, la implementación o el hardware sean iguales.

B. *Verdadero.* Big O describe cómo cambia el costo de un algoritmo cuando aumenta el tamaño de los datos. No indica necesariamente el tiempo exacto en segundos.

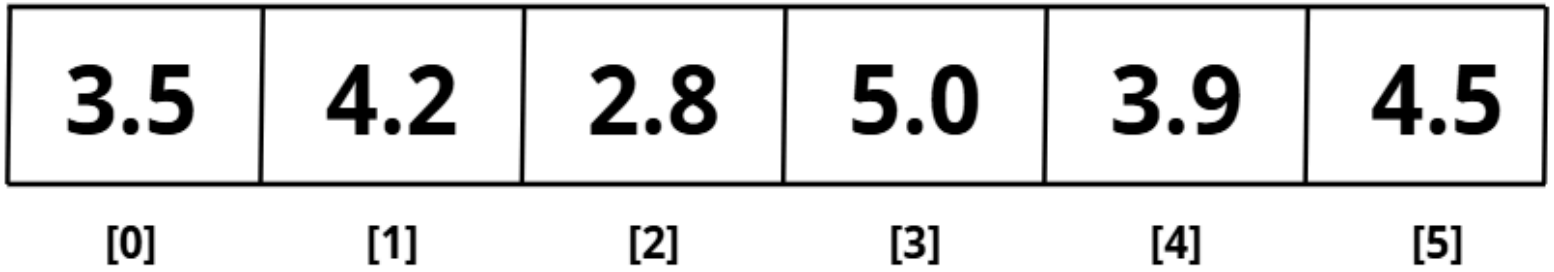
C. *Falso.* En un arreglo, acceder a **arreglo[5]** es **O(1)**, porque el índice permite calcular directamente la ubicación del elemento. No es necesario recorrer los cinco elementos anteriores.

Parte 2 — Arreglos 1D

2.1 Diseña el arreglo

1. Dibujo del arreglo notas

Los arreglos comienzan en el índice 0.



Índices: 0, 1, 2, 3, 4 y 5.

2. Tercera nota ingresada

La tercera nota es 2.8 y está en el índice 2, porque el conteo de posiciones comienza en cero:

- **Primera nota:** índice 0 → 3.5.
- **Segunda nota:** índice 1 → 4.2.
- **Tercera nota:** índice 2 → 2.8.

3. Acceder a la última nota

- notas[5]

También se puede escribir de forma general:

- notas[longitud(notas) - 1]

Esto permite acceder directamente a la última posición sin recorrer todo el arreglo.

2.2 Direcciones de memoria

Datos:

- **Dirección base:** 0x2000
- **Tamaño de cada valor:** 4 bytes.
- **Fórmula:** dirección = base + (índice × tamaño)

1. Cálculos

Elemento	Cálculo	Dirección
notas[0]	0x2000 + (0 × 4)	**0x2000**
notas[3]	0x2000 + (3 × 4)	**0x200C**
notas[5]	0x2000 + (5 × 4)	**0x2014**

En hexadecimal, **4 bytes** equivalen a **0x4**, así que la dirección aumenta cuatro bytes por cada índice.

2. ¿Por qué acceder a notas[3] es igual de rápido que a notas[0]?

Porque el arreglo guarda sus elementos en posiciones consecutivas de memoria. Para acceder a cualquier índice, el sistema calcula la dirección usando la base, el índice y el tamaño de cada elemento. No necesita revisar los elementos anteriores, por eso el acceso directo es **O(1)**.

2.3 Búsqueda lineal vs. binaria

Arreglo:

edades = [15, 18, 20, 23, 27, 31, 35, 40]

1. Búsqueda lineal de 31

Se revisan los elementos desde el principio hasta encontrar el valor:

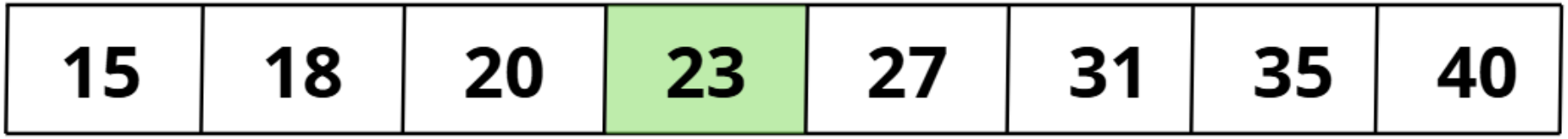
Comparación	índice	Valor	¿Es 31?
1	0	15	No
2	1	18	No
3	2	20	No
4	3	23	No
5	4	27	No
6	5	31	Si

Resultado: 6 comparaciones. Se encontró el valor en el índice 5.

2. Búsqueda binaria de 31

Se aprovecha que el arreglo está ordenado y se descarta la mitad que no puede contener el valor.

Comparación 1:



Centro: 23. Como 31 es mayor, se busca en la mitad derecha.

Comparación 2:



Centro de la mitad derecha: 31. Se encontró el valor.

Resultado: 2 comparaciones. El valor está en el índice 5.

3. ¿Por qué la búsqueda binaria necesita un arreglo ordenado?

Porque decide qué mitad descartar comparando el valor buscado con el elemento central, si el arreglo no está ordenado, no se puede saber si el valor está a la izquierda o a la derecha, así que descartar una mitad podría eliminar el elemento que buscamos.

4. ¿Cuál usaría con 1 millón de elementos?

Usaría la búsqueda binaria, siempre que el arreglo esté ordenado y pueda accederse eficientemente a sus posiciones.

- **Lineal:** O(n), puede revisar hasta 1 millón de elementos.

- **Binaria:** O(log n), descarta aproximadamente la mitad en cada paso y necesita alrededor de 20 comparaciones para un millón de elementos en el peor caso ideal.

La diferencia aumenta mucho cuando el tamaño de los datos crece.

2.4 Insertar un elemento

Arreglo inicial:

edades = [15, 18, 20, 23, 27]

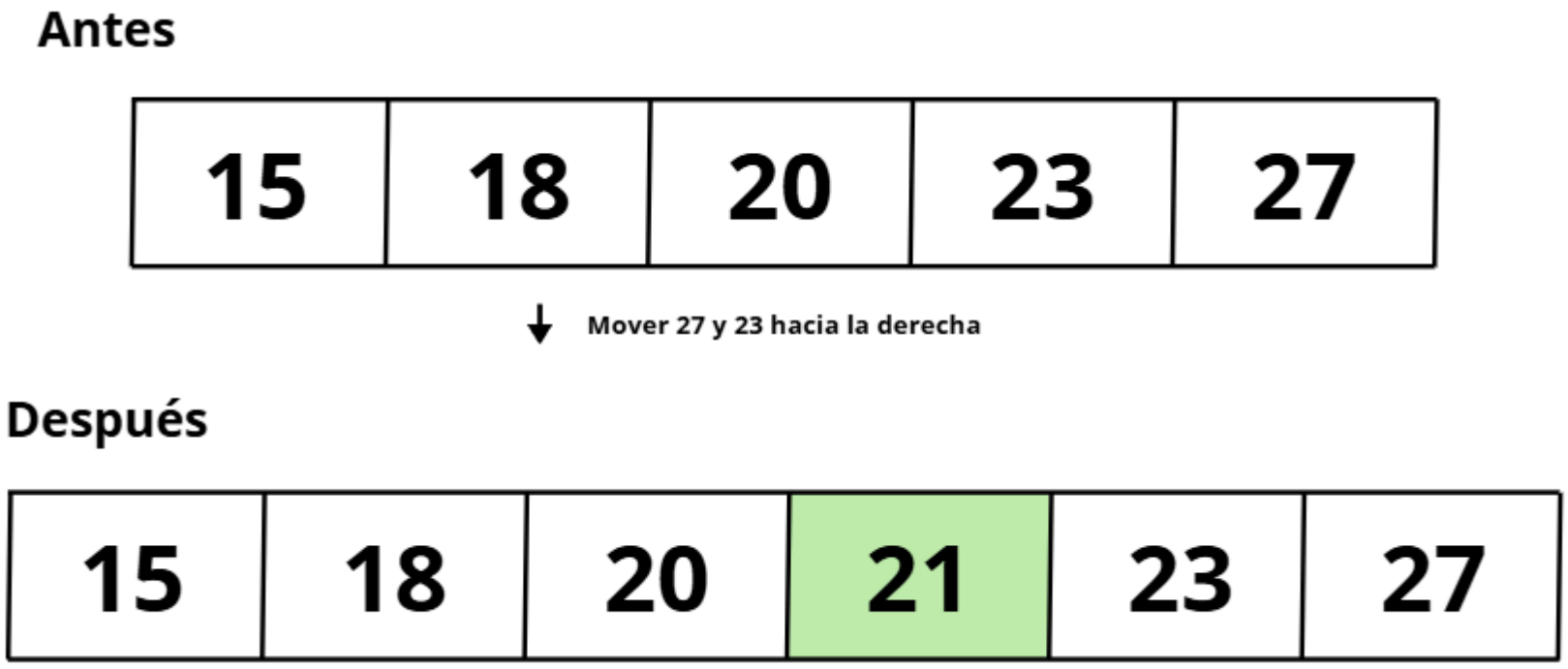
1. Insertar al final

- **Si hay espacio disponible:** *O(1)*, porque se coloca el nuevo valor en la siguiente casilla libre.
- **Si no hay espacio disponible:** *O(n)*, porque se necesita crear un arreglo más grande y copiar los elementos anteriores.

En un arreglo de tamaño fijo, no se puede ampliar directamente sin reservar un espacio nuevo.

2. Insertar el valor 21 en la mitad

Para conservar el orden, primero hay que mover los elementos mayores que 21 una posición hacia la derecha.



Se deben desplazar 27 y 23 para abrir un espacio en el índice 3. Después se coloca 21 en esa posición.

Complejidad: O(n), porque en el peor caso es necesario mover muchos elementos.

3. Comparación

Operación	**Complejidad**
Insertar al final con espacio	O(1)
Insertar al final sin espacio	O(n)
Insertar en la mitad conservando el orden	O(n)

Parte 3 — Arreglos 2D

3.1 Diseña la matriz

1. Dibujo de la matriz

La matriz **salón** tiene 3 filas y 4 columnas. Los índices comienzan en cero.

F\C	0	1	2	3
0	[0][0]	[0][1]	[0][2]	[0][3]
1	[1][0]	[1][1]	[1][2]	[1][3]
2	[2][0]	[2][1]	[2][2]	[2][3]

Cada casilla puede almacenar un **0** *(puesto libre)* o un **1** *(puesto ocupado)*.

2. Acceder al puesto de la fila 2, columna 3

- salon[2][3]

Primero se selecciona la fila 2 y luego la columna 3 dentro de esa fila.

3. Cantidad total de casillas

La matriz tiene:
3 × 4 = 12 casillas

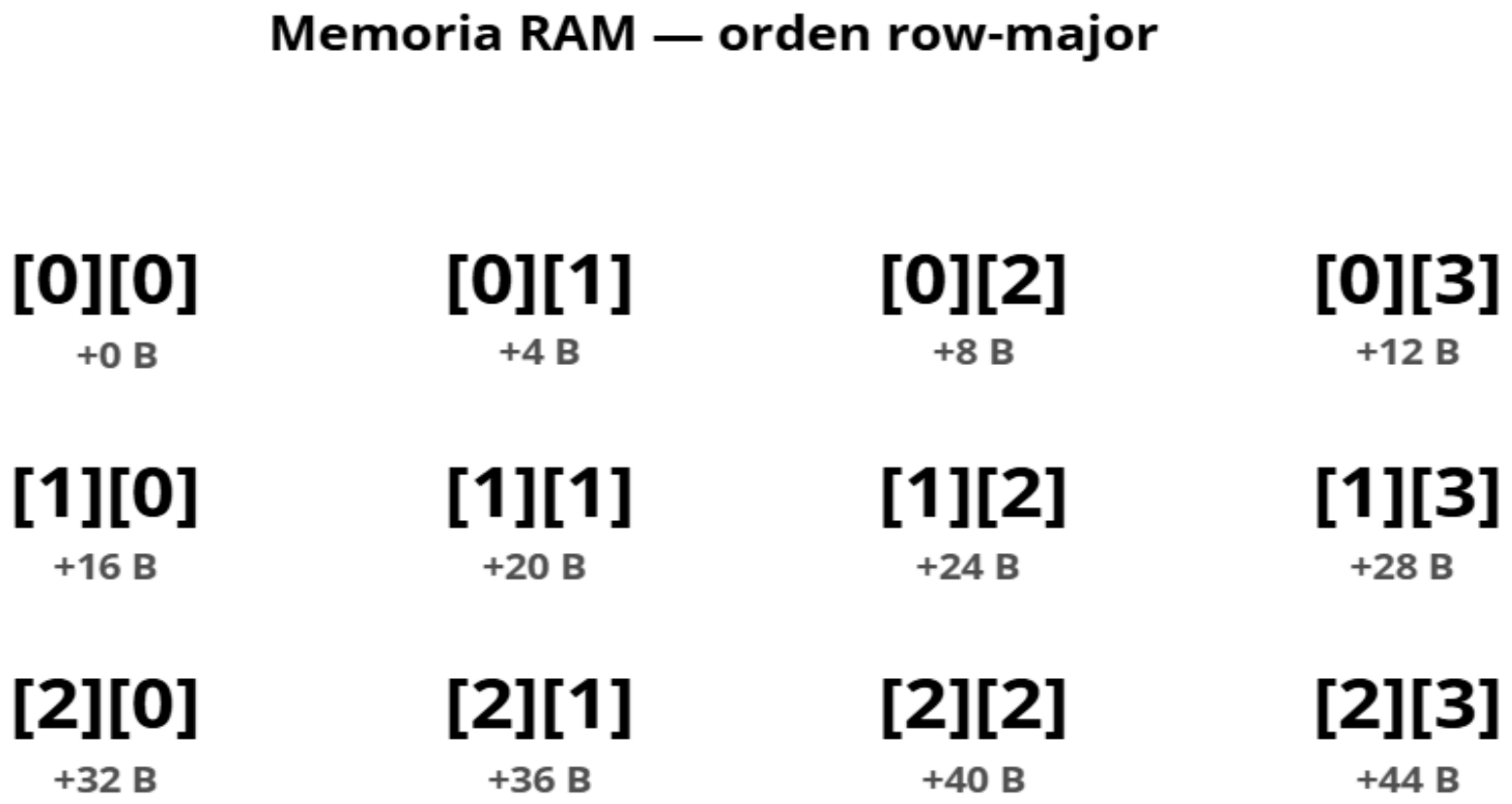
En general:
Total de casillas = número de filas × número de columnas.

3.2 De 2D a memoria (row-major)

Datos:

- **Dirección base:** 0x1000.
- 3 filas × 4 columnas.
- Cada valor ocupa 4 bytes.
- Se almacena fila por fila.

1. Las 12 casillas aplanadas



2. Dirección de [1][2]

Fórmula:

dirección = base + ((fila × columnas + columna) × tamaño)

Reemplazando:

0x1000 + ((1 × 4 + 2) × 4)

= 0x1000 + (6 × 4)

= 0x1000 + 24

Dirección: 0x1018.

3. Direcciones de [0][0] y [2][3]

Casilla	Cálculo	Dirección
[0][0]	0x1000 + ((0 × 4 + 0) × 4)	0x1000
[2][3]	0x1000 + ((2 × 4 + 3) × 4)	0x102C

3.3 Recorrido con ciclos anidados

1. Pseudocódigo

```
ocupados ← 0

        PARA fila DESDE 0 HASTA f - 1:
                PARA columna DESDE 0 HASTA c - 1:
                        SI salon[fila][columna] = 1:
                                ocupados ← ocupados + 1
MOSTRAR ocupados
```

El ciclo exterior recorre las filas y el interior revisa cada columna de la fila actual. Cada vez que encuentra un **1**, incrementa el contador.

2. Complejidad

O(f × c).

El ciclo exterior se ejecuta f veces y, por cada fila, el ciclo interior se ejecuta c veces. Por eso se revisan f × c casillas.

Si se considera una matriz cuadrada de n × n, la complejidad se expresa como **O(n²)**.

3. ¿Qué pasa si se duplican f y c?

El número de casillas pasa de f × c a:

(2f) × (2c) = 4fc

Por lo tanto, el tiempo de ejecución aproximadamente se cuadruplica, porque hay cuatro veces más casillas que revisar.

3.4 Conectando todo

Big O nos permite entender cómo aumenta el tiempo de un algoritmo cuando crecen los datos. Los arreglos 1D organizan elementos en una sola dimensión y permiten acceder a ellos directamente mediante un índice. Los arreglos 2D organizan la información en filas y columnas, por lo que son útiles para representar salones, tablas o matrices. Entender la complejidad ayuda a elegir cómo recorrer y buscar los datos. Por ejemplo, la búsqueda lineal puede ser suficiente para pocos elementos, pero la binaria es más eficiente en arreglos ordenados grandes. Así, las estructuras de datos y Big O se complementan para resolver problemas de manera eficiente.